

P0009R7

mdspan: A Non-Owning Multidimensional Array Reference

Draft Proposal, 24 June 2018

This version:

wg21.link/P0009r7

Issue Tracking:

[GitHub](https://github.com)

Authors:

[H. Carter Edwards](#)

[Bryce Adelstein Lelbach](#)

[Daniel Sunderland](#)

[David Hollman](#)

[Christian Trott](#)

[Mauro Bianco](#)

[Ben Sander](#)

[Athanasios Iliopoulos](#)

[John Michopoulos](#)

[Mark Hoemmen](#)

Audience:

LWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Source:

github.com/ORNL/cpp-proposals-pub/blob/master/P0009/P0009.bs

Table of Contents

1	Revision History
1.1	P0009r7: Post 2018-06-Rapperswil Mailing
1.2	P0009r6 : Pre 2018-06-Rapperswil Mailing
1.3	P0009r5 : Pre 2018-03-Jacksonville Mailing
1.4	P0009r4 : Pre 2017-11-Albuquerque Mailing
1.5	P0009r3 : Post 2016-06-Oulu Mailing
1.6	P0009r2 : Pre 2016-06-Oulu Mailing
1.7	P0009r1 : Pre 2016-02-Jacksonville Mailing
1.8	P0009r0 : Pre 2015-10-Kona Mailing
1.9	Related Activity
2	Description
3	Editing Notes
4	Wording
5	Next Steps
6	Related Work
7	TODO

References

Informative References

§ 1. Revision History

§ 1.1. P0009r7: Post 2018-06-Rapperswil Mailing

- wording reworked based on guidance: [LWG review at 2018-06-Rapperswil](#)
- usage of `span` requires reference to C++20 working draft
- namespace for library TS `std::experimental::fundamentals_v3`

§ 1.2. P0009r6 : Pre 2018-06-Rapperswil Mailing

P0009r5 was not taken up at 2018-03-Jacksonville meeting. Related [LEWG review of P0900 at 2018-03-Jacksonville meeting](#)

LEWG Poll: We want the ability to customize the access to elements of `span` (ability to restrict, etc):

```
span<T, N, Accessor=...>
```

SF	F	N	A	SA
1	1	1	2	8

LEWG Poll: We want the customization of `basic_mdspan` to be two concepts `Mapper` and `Accessor` (akin to `Allocator` design).

```
basic_mdspan<T, Extents, Mapper, Accessor>
mdspan<T, N...>
```

SF	F	N	A	SA
3	4	5	1	0

LEWG Poll: We want the customization of `basic_mdspan` to be an arbitrary (and potentially user-extensible) list of properties.

```
basic_mdspan<T, Extents, Properties...>
```

SF	F	N	A	SA
1	2	2	6	2

Changes from P0009r5 due to related LEWG reviews:

- Replaced variadic property list with *extents*, *layout mapping*, and *accessor* properties.
- Incorporated [P0454r1](#).
 - Added accessor policy concept.
 - Renamed `mdspan` to `basic_mdspan`.
 - Added a `mdspan` alias to `basic_mdspan`.

§ 1.3. P0009r5 : Pre 2018-03-Jacksonville Mailing

[LEWG review of P0009r4 at 2017-11-Albuquerque meeting](#)

LEWG Poll: We should be able to index with `span<int type[N]>` (in addition to array).

SF	F	N	A	SA
2	11	1	1	0

Against comment - there is not a proven needs for this feature.

LEWG Poll: We should be able to index with 1d `mdspan`.

SF	F	N	A	SA
0	8	7	0	0

LEWG Poll: We should put the requirement on "rank() <= N" back to "rank()==N".

Unanimous consent

LEWG Poll: With the editorial changes from small group, plus the above polls, forward this to LWG for Fundamentals v3.

Unanimous consent

Changes from P0009r4:

- Removed nullptr constructor.
- Added constexpr to indexing operator.
- Indexing operator requires that `rank()==sizeof...(indices)`.
- Fixed typos in examples and moved them to appendix.
- Converted note on how extentions to access properties may cause reference to be a proxy type to an "see below" to make it normative.

§ 1.4. P0009r4 : Pre 2017-11-Albuquerque Mailing

[LEWG review at 2017-03-Kona meeting](#)

[LEWG review of P0546r1 at 2017-03-Kona meeting](#)

LEWG Poll: Should we have a single template that covers both single and multi-dimensional spans?

SF	F	N	A	SA
1	6	2	6	3

Changes from P0009r3:

- Align with P0122r5 `spanproposal`.
- Rename to `mdspan`, multidimensional span, to align with `span`.
- Move preferred array extents mechanism to appendix.
- Expose codomain as a `span`.
- Add layout mapping concept.

§ 1.5. P0009r3 : Post 2016-06-Oulu Mailing

[LEWG review at 2016-06-Oulu](#)

LEWG did not like the name `array_ref`, and suggested the following alternatives: - `sci_span` - `numeric_span` - `multidimensional_span` - `multidim_span` - `mdspan` - `md_span` - `vla_span` - `multispan` - `multi_span`

LEWG Poll: Are member `begin()/end()` still good?

SF	F	N	A	SA
0	2	4	3	1

LEWG Poll: Want this proposal to provide range-producing functions outside `array_ref`?

SF	F	N	A	SA
0	1	3	2	3

LEWG Poll: Want a separate proposal to explore iteration design space?

SF	F	N	A	SA
9	1	0	0	0

Changes from P0009r2:

- Removed iterator support; a future paper will be written on the subject.
- Noted difference between multidimensional array versus language's array-of-array-of-array...
- Clearly describe requirements for the embedded type aliases ([element_type](#), [reference](#), etc).
- Expanded description of how the variadic properties list would work.
- Stopped allowing `array_ref<T[N]>` in addition to `array_ref<extents<N>>`.
- Clarified domain, codomain, and domain -> codomain mapping specifications.
- Consistently use *extent* and *extents* for the multidimensional index space.

1.6. P0009r2 : Pre 2016-06-Oulu Mailing

LEWG review at 2016-02-Jacksonville.

Changes from P0009r1:

- Adding details for extensibility of layout mapping.
- Move motivation, examples, and relaxed incomplete array type proposal to separate papers.
 - [P0331: Motivation and Examples for Polymorphic Multidimensional Array.](#)
 - [P0332: Relaxed Incomplete Multidimensional Array Type Declaration.](#)

1.7. P0009r1 : Pre 2016-02-Jacksonville Mailing

LEWG review at 2015-10-Kona.

LEWG Poll: What should this feature be called?

Name	#
view	5
span	9
array_ref	6
slice	6
array_view	6
ref	0
array_span	7
basic_span	1
object_span	3
field	0

LEWG Poll: Do we want 0-length static extents?

SF	F	N	A	SA
3	4	2	3	0

LEWG POLL: Do we want the language to support syntaxes like `X[3][][5]`?

Syntax	#
<code>view<int[3][0][5], property1></code>	12
<code>view<int, dimension<3, 0, dynamic_extent, 5>, property1></code>	4
<code>view<int[3][0][dynamic_extent][5], property1></code>	5
<code>view<int, 3, 0, dynamic_extent, 5, property1></code>	4

<code>view<int, 3, 0, dynamic_extent, 5, properties<property1>></code>	2
<code>view<arr<int, 3, 0, dynamic_extent, 5>, property1></code>	4
<code>view<int[3][0][][5], properties<property1>></code>	9

LEWG POLL: Do we want the variadic property list in template args (either raw or in `properties<>`)? Note there is no precedence for this in the library.

SF	F	N	A	SA
3	6	3	0	0

LEWG POLL: Do we want the per-view bounds-checking knob?

SF	F	N	A	SA
3	4	1	2	1

Changes from P0009r0:

- Renamed `view` to `array_ref`.
- How are users allowed to add properties? Needs elaboration in paper.
- `view<int[ ][ ][ ]>::layout` should be named.
- Rename `is_regular` (possibly to `is_affine`) to avoid overloading the term with the `Regular` concept.
- Make static `span()`, `operator()`, constructor, etc variadic.
- Demonstrate the need for improper access in the paper.
- In `operator()`, take integral types by value.

§ 1.8. P0009r0 : Pre 2015-10-Kona Mailing

Original non-owning multidimensional array reference (`view`) paper with motivation, specification, and examples.

§ 1.9. Related Activity

Related [LEWG review of P0546r1 at 2017-11-Albuquerque meeting](#)

LEWG Poll: `span` should specify the dynamic extent as the element type of the first template parameter rather than the (current) second template parameter

SF	F	N	A	SA
5	3	2	2	0

LEWG Poll: `span` should support the addition of access properties variadic template parameters

SF	F	N	A	SA
0	10	1	5	0

Authors agreed to bring a separate paper ([\[P0900r0\]](#)) discussing how the variadic properties will work.

§ 2. Description

The proposed polymorphic multidimensional array reference (`mdspan`) defines types and functions for mapping indices from the **domain**, a **multidimensional index space**, to the **codomain**, elements of a contiguous span of objects. A multidimensional index space is defined as the Cartesian product of integer extents, $[0..N_0) * [0..N_1) * [0..N_2) \dots$. An `mdspan` has two policies: the **layout mapping** and the **accessor**. The layout mapping specifies the formula, and properties of the formula, for mapping a multi-index from the domain to an element in the codomain. The accessor is an extension point that allows modification of how elements are accessed. For example, the Accessors paper (P0367) proposed a rich set of potential access properties.

A multidimensional array is not an array-of-array-of-array-of-array...

The multidimensional array abstraction has been fundamental to numerical computations for over five decades. However, the C/C++ language provides only a one-dimensional array abstraction which can be composed into array-of-array-of-array... types. While such types have some similarity to multidimensional arrays, they do not provide adequate multidimensional array functionality of this proposal. Two critical functionality differences are (1) multiple dynamic extents and (2) polymorphic mapping of multi-indices to element objects.

Optimized Implementation of Layout Mapping

The layout mapping of a multi-index is intended to be an $O(1)$ `constexpr` operation that is trivially inlined and optimized. Note that Fortran compilers' optimizations include loop invariant code motion, including partial evaluation of multi-index layout mappings when indices are loop-invariant.

§ 3. Editing Notes

The proposed changes are relative to the working draft of the standard as of [N4750](#).

The  character is used to denote a placeholder section number, table number, or paragraph number which the editor shall determine.

Add the header `<mdspan>` to table 16 in **[headers]**.

Add the header `<mdspan>` to Table 76 in 26.1 **[containers.general]** below the listing for `<span>`.

§ 4. Wording

Text in blockquotes is not proposed wording

The  character is used to denote a placeholder section number which the editor shall determine.

Copy **[views]** *subclause* (currently 26.7 in *n4750*) which contains the wording for `span` and `dynamic_extent` which is needed for `basic_mdspan`

◇ ◇ ◇

*Add the following paragraphs to *[views.general]:*

◇. The header `<mdspan>` defines the view `basic_mdspan`, the type alias `mdspan`, and other facilities for interacting with these views. The `basic_mdspan` class template maps a multi-index within a multi-index *domain* to a reference an element in the *codomain* `span`.

◇. The `subspan` function generates a `basic_mdspan` with a domain contained within the input `basic_mdspan` domain and codomain contained within the input `basic_mdspan` codomain.

◇ ◇ ◇

*Add the following subclauses to the end of the **[views]** subclause (currently 26.7 in *n4750*):*

26.7.◇ Header `<mdspan>` synopsis [mdspan.syn]

```

namespace std {
namespace experimental {
namespace fundamentals_v3 {
    // [mdspan.extents], class template extents
    template<ptrdiff_t... StaticExtents>
        class extents;

    // [mdspan.layout], Layout mapping policies
    class layout_left;
    class layout_right;
    class layout_stride;

    // [mdspan.accessor.basic], class template accessor_basic
    template<class ElementType>
        class accessor_basic;

    // [mdspan.basic], class template mdspan
    template<class ElementType,
            class Extents,
            class LayoutPolicy = layout_right,
            class Accessor = accessor_basic<ElementType>>
        class basic_mdspan;

    template<class T, ptrdiff_t... Extents>
        using mdspan = basic_mdspan<T, extents<Extents...>>;

    // [mdspan.extents.compare], extents comparison operators
    template<ptrdiff_t... LHS, ptrdiff_t... RHS>
        constexpr bool operator==(const extents<LHS...>& lhs, const extents<RHS...>& rhs) noexcept;
    template<ptrdiff_t... LHS, ptrdiff_t... RHS>
        constexpr bool operator!=(const extents<LHS...>& lhs, const extents<RHS...>& rhs) noexcept;

    // [mdspan.subspan], subspan creation
    template<class ElementType, class Extents, class LayoutPolicy,
            class Accessor, class... SliceSpecifiers>
        basic_mdspan<ElementType, /* see-below */>
        subspan(const basic_mdspan<ElementType, Extents, LayoutPolicy, Accessor>&, SliceSpecifiers...) noexcept;

    // tag supporting subspan
    struct all_type { explicit all_type() = default; };
    inline constexpr all_type all = all_type{};
}}}

```

26.7. ♦ Class template `extents` [mdspan.extents]

26.7. ♦.1 Overview [mdspan.extents.overview]

1. An `extents` object defines a *multidimensional index space* which is the Cartesian product of integers `extents [0..N0) * [0..N1) * ...`
2. The *dynamic extents* of an `extents` object correspond to the `StaticExtents` template parameters that are equal to `dynamic_extent`. Let *DynamicRank*[*i*] denote the index of the *i*th such extent in the `StaticExtents` template parameter pack, and let *DynamicIndex*[*r*] indicate the number of such extents in the first *r* entries of the `StaticExtents` parameter pack
3. An `extents` object is expected to store dynamic extents. [Note: An implementation should not consume storage for static extents. — end note]
4. If any of `StaticExtents` are negative and not equal to `dynamic_extent`, the program is ill-formed.

```

namespace std {
namespace experimental {
namespace fundamentals_v3 {

template<ptrdiff_t... StaticExtents>
class extents {
public:
    // types
    using index_type = ptrdiff_t;

    // [mdspan.extents.cons], Constructors and assignment
    constexpr extents() noexcept;
    constexpr extents(const extents&) noexcept;
    constexpr extents(extents&&) noexcept;
    template<class... IndexType>
    constexpr extents(IndexType... dynamic_extents) noexcept;
    template<class IndexType, size_t rank_dynamic>
    constexpr extents(const array<IndexType, rank_dynamic>&) noexcept;
    template<ptrdiff_t... OtherStaticExtents>
    constexpr extents(const extents<OtherStaticExtents...>& other);
    ~extents() = default;

    constexpr extents& operator=(const extents&) noexcept = default;
    constexpr extents& operator=(extents&&) noexcept = default;
    template<ptrdiff_t... OtherStaticExtents>
    constexpr extents& operator=(const extents<OtherStaticExtents...>& other);

    // [mdspan.extents.obs], Observers of the domain multi-index space
    static constexpr size_t rank() noexcept;
    static constexpr size_t rank_dynamic() noexcept;
    static constexpr index_type static_extent(size_t) noexcept;
    constexpr index_type extent(size_t) const noexcept;

private:
    array<index_type, rank_dynamic()> dynamic_extents_; // exposition only
};

}}}
```

26.7.2 Constructors and assignment [mdspan.extents.cons]

```
constexpr extents() noexcept;
```

- *Effects*: Aggregate-initializes `dynamic_extents_` to `{ }`
- *Postconditions*: `extent(r)` if `static_extent(r)==dynamic_extent` for all `r` in the range `[0, rank())`

```
constexpr extents(const extents& other);
constexpr extents(extents&& other);
```

- *Effects*: Initializes `dynamic_extents_` with `other.dynamic_extents_`
- *Postconditions*: `extent(r)==other.extent(r)` for all `r` in the range `[0, rank())`

```
template<ptrdiff_t... OtherStaticExtents>
constexpr extents(const extents<OtherStaticExtents...>& other);
```

- *Requires*: For each `r` in the range `[0, rank())`, if `static_extent(r)!=dynamic_extent`, then `static_extent(r)==other.extent(r)`.

- *Effects*: For each `r` in the range `[0, rank())`, if `static_extent(r)==dynamic_extent`, initializes `dynamic_extents[DynamicRank[r]]` with `other.extent(r)`.
- *Throws*: Nothing.
- *Postconditions*: `*this==other`
- *Remarks*: This constructor shall not participate in overload resolution unless `sizeof...(StaticExtents)==sizeof...(OtherStaticExtents)`.

```
template<class... IndexType>
constexpr extents(IndexType... dynamic_extents) noexcept;
```

- *Requires*: `((dynamic_extents>=0) && ...)`
- *Effects*: Aggregate-initializes `dynamic_extents_` to `{dynamic_extents...}`
- *Postconditions*: `extent(DynamicRank[i])` is equal to the `i`th entry in the parameter pack `dynamic_extents`
- *Remarks*: This constructor shall not participate in overload resolution unless:
 - `(is_convertible_v<IndexType, index_type> && ...)`
 - and `sizeof...(dynamic_extents)==rank_dynamic()`

```
template<class IndexType, size_t rank_dynamic>
constexpr extents(const array<IndexType, rank_dynamic> & dynamic_extents) noexcept;
```

- *Requires*: `dynamic_extents[i]>=0` for all `i` where `0 <= i < rank_dynamic()`
- *Effects*: Initializes `dynamic_extents_` with `dynamic_extents`
- *Postconditions*: `extent(DynamicRank[i])` is equal to `dynamic_extents[i]`
- *Remarks*: This constructor shall not participate in overload resolution unless:
 - `is_convertible_v<IndexType, index_type>`

```
template<ptrdiff_t... OtherStaticExtents>
constexpr extents& operator=(const extents<OtherStaticExtents...>& other);
```

- *Requires*: For each `r` in the range `[0, rank())`, if `static_extent(r)!=dynamic_extent`, then `static_extent(r)==other.extent(r)`.
- *Effects*: For each `r` in the range `[0, rank())`, if `static_extent(r)==dynamic_extent`, assigns `dynamic_extents[DynamicRank[r]]` to `other.extent(r)`.
- *Throws*: Nothing.
- *Postconditions*: `*this==other`
- *Returns*: `*this`.
- *Remarks*: This constructor shall not participate in overload resolution unless `sizeof...(StaticExtents)==sizeof...(OtherStaticExtents)`.

26.7.3 Observers of the domain multi-index space [mdspan.extents.obs]

```
static constexpr size_t rank() const noexcept;
```

- *Returns*: `sizeof...(StaticExtents)`

```
static constexpr size_t rank_dynamic() const noexcept;
```

- **Returns:** `((StaticExtents==dynamic_extent)+...)` [Note: This is the number of dynamic extents — end note]

```
static constexpr index_type static_extent(size_t r) const noexcept;
```

- **Returns:** The `r`th entry in the `StaticExtents` parameter pack if $0 \leq r < \text{rank}()$, or 1 otherwise.

```
constexpr index_type extent(size_t r) const noexcept;
```

- **Returns:**
 - If `static_extent(r)==dynamic_extent`, then `dynamic_extents_[DynamicRank[r]]`.
 - Otherwise, `static_extent(r)`.

26.7.4 extents comparison operators [mdspan.extents.compare]

```
template<ptrdiff_t... LHS, ptrdiff_t... RHS>
constexpr bool operator==(const extents<LHS...>& lhs, const extents<RHS...>& rhs) noexcept;
t;
```

- **Returns:** true if `lhs.rank()==rhs.rank()` and `lhs.extents(r)==rhs.extents(r)` for all `r` in the range `[0, lhs.rank())`, or false otherwise.

```
template<ptrdiff_t... LHS, ptrdiff_t... RHS>
constexpr bool operator!=(const extents<LHS...>& lhs, const extents<RHS...>& rhs) noexcept;
t;
```

- **Returns:** `!(lhs==rhs)`

26.7. Layout mapping policy [mdspan.layout]

26.7.1 Layout mapping requirements [mdspan.layout.reqs]

1. A *layout mapping policy* is a class that contains a *layout mapping*, a nested class template.
2. A *layout mapping policy* shall meet the requirements in table 4.
3. A *layout mapping* shall meet the requirements of `DefaultConstructible`, `CopyAssignable`, `EqualityComparable`, and the requirements in table 4.
4. In Table 4:
 - `MP` denotes a layout mapping policy.
 - `E` denotes a specialization of `extents`.
 - `e` denotes an object of type `E` defining a domain multi-index space.
 - `r` is a value of an integral type such that $0 \leq r < e.\text{rank}()$.
 - `i...` and `j...` are packs of an integer type denoting values in the multi-index space `e`, the `r`*th member of packs `i...` and `j...` are denoted by `i[r]` and `j[r]`, and `sizeof...(i)==E::rank()`, 0

$\leq i[r] < e.extent(r)$, $sizeof... (j) == E::rank()$, and $0 \leq j[r] < e.extent(r)$.

- `M` denotes a layout mapping class.
- `m` denotes an object of type `M` that maps a multi-index `i...` to an integral value.

Table 26.7.2 — Layout mapping policy and layout mapping requirements

Expression	Return Type	Operational Semantics	Requires/Remarks
<code>MP::template mapping<E></code>	<code>M</code>		
<code>m.get_extents()</code>	<code>E</code>	Returns: <code>e</code> .	
<code>m(i...)</code>	<code>E::index_type</code>	Returns: Mapping of a multi-index <code>i...</code>	Requires: $0 \leq m(i...)$
<code>m.required_span_size()</code>	<code>E::index_type</code>	Returns: one plus the maximum value of <code>m(i...)</code> .	
<code>m.is_unique()</code>	<code>bool</code>	Returns: true if $m(i...) \neq m(j...)$ for every $i... \neq j...$	
<code>m.is_contiguous()</code>	<code>bool</code>	Returns: true if the set of values defined by <code>m(i)...</code> is equal to the set of values consisting of $0 \dots m.required_span_size() - 1$	
<code>m.is_strided()</code>	<code>bool</code>	Returns: true if $m(j...) - m(i...) = s[r]$ when all members of <code>j...</code> and <code>i...</code> are equal except for exactly one <i>r</i> th member such that $j[r] == i[r] + 1$. <code>s[r]</code> is the stride of ordinate <i>r</i> .	
<code>M::is_always_unique()</code>	<code>bool</code>	Returns: true if <code>m.is_unique() == true</code> for any object of type <code>M</code> .	
<code>M::is_always_contiguous()</code>	<code>bool</code>	Returns: true if <code>m.is_contiguous() == true</code> for any object of type <code>M</code> .	
<code>M::is_always_strided()</code>	<code>bool</code>	Returns: true if <code>m.is_strided() == true</code> for any object of type <code>M</code> .	
<code>m.stride(r)</code>	<code>E::index_type</code>	Returns: $m(j...) - m(i...)$ when all members of <code>j...</code> and <code>i...</code> are equal except for the <i>r</i> th member such that $j[r] == i[r] + 1$.	Requires: <code>m.is_strided() == true</code>

26.7.2 Class `layout_left` [mdspan.layout.left]

1. `layout_left` meets the requirements of layout mapping policy. [Note: Thus, any well-formed specialization of `layout_left::template mapping` meets the requirements of layout mapping —end note]
2. `layout_left` gives a layout mapping where the left-most extent is stride one and strides increase left-to-right as the product of extents.
3. If `Extents` is not a (possibly cv-qualified) specialization of `extents`, the program is ill-formed.

```

namespace std {
namespace experimental {
namespace fundamentals_v3 {

struct layout_left {
    template<class Extents>
    class mapping {
    public:
        // [mdspan.layout.left.cons], layout_left::mapping constructors
        constexpr mapping() noexcept;
        constexpr mapping(const mapping& other) noexcept;
        constexpr mapping(mapping&& other) noexcept;
        constexpr mapping(const Extents& e) noexcept;
        template<class OtherExtents>
            constexpr mapping(const mapping<OtherExtents>& other);

        mapping& operator=() noexcept = default;
        mapping& operator=(const mapping& other) noexcept = default;
        template<class OtherExtents>
            constexpr mapping& operator=(const mapping<OtherExtents>& other);

        // [mdspan.layout.left.ops], layout_left::mapping operations
        Extents get_extents() const noexcept;

        constexpr typename Extents::index_type required_span_size() const noexcept;

        template<class... Indices>
            typename Extents::index_type operator()(Indices... is) const;

        static constexpr bool is_always_unique();
        static constexpr bool is_always_contiguous();
        static constexpr bool is_always_strided();

        constexpr bool is_unique() const;
        constexpr bool is_contiguous() const;
        constexpr bool is_strided() const;

        template<class OtherExtents>
            constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
        template<class OtherExtents>
            constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;

        typename Extents::index_type stride(size_t rank) const noexcept;

    private:
        Extents extents_; // exposition only
    };
};

}}}

```

26.7.2.1 `layout_left::mapping` constructors [mdspan.layout.left.cons]

```
constexpr mapping() noexcept;
```

- *Effects:* Default-initializes `extents_`.
- *Postconditions:* `get_extents() == Extents()`

```
constexpr mapping(const mapping& other) noexcept;
```

- *Effects*: Initializes `extents_` with `other.extents_`.
- *Postconditions*: `get_extents() == other.get_extents()`.

```
constexpr mapping(mapping&& other) noexcept;
```

- *Effects*: Initializes `extents_` with `move(other.extents_)`.
- *Postconditions*: `get_extents()` returns a copy of an `Extents` that is equal to the copy returned by `other.get_extents()` before the invocation of the move.

```
constexpr mapping(const Extents & e) noexcept;
```

- *Effects*: Initializes `extents_` with `e`.
- *Postconditions*: `get_extents() == e`.

```
template<class OtherExtents>
constexpr mapping(const mapping<OtherExtents>& other);
```

- *Requires*: `other.get_extents()` meets the requirements for use in the initialization of `extents_`.
- *Effects*: Initializes `extents_` with `other.get_extents()`.
- *Postconditions*: `get_extents() == other.extents_`.
- *Throws*: nothing.

26.7.2.2 `layout_left::mapping` operations [mdspan.layout.left.ops]

```
Extents get_extents() const noexcept;
```

- *Returns*: `extents_`.

```
typename Extents::index_type required_span_size() const noexcept;
```

- *Returns*: The product of `get_extents().extent(r)` for all `r` where $0 \leq r < \text{get_extents().rank}()$

```
template<class... Indices>
typename Extents::index_type operator()(Indices... i) const;
```

Let `i[k]` denote the `k`th member of `i...`.

- *Returns*: Equivalent to `offset` in

```
Extents::index_type offset = 0 ;
for(size_t k=0; k<get_extents().rank(); ++k)
    offset += i[k]*stride(k);
```

- *Remarks:* This operator shall not participate in overload resolution unless
 - `sizeof...(Indices)==get_extents().rank()`,
 - and `is_convertible_v<Indices, typename Extents::index_type> && ...`

```
static constexpr bool is_always_unique();
static constexpr bool is_always_contiguous();
static constexpr bool is_always_strided();
constexpr bool is_unique() const;
constexpr bool is_contiguous() const;
constexpr bool is_strided() const;
```

- *Returns:* `true`

```
typename Extents::index_type stride(size_t r) const
```

- *Returns:* Equivalent to `s` in

```
Extents::index_type s = 1;
for(size_t k=0; k<r; ++k)
    s *= get_extents().extent(k);
```

```
template<class OtherExtents>
constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
```

- *Returns:* `get_extents()==other.get_extents()`.

```
template<class OtherExtents>
constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;
```

- *Returns:* `get_extents() != other.get_extents()`.

26.7.3 Class `layout_right` [mdspan.layout.right]

1. `layout_right` meets the requirements of layout mapping policy. *[Note: Thus, any well-formed specialization of `layout_right::template mapping` meets the requirements of layout mapping —end note]*
2. The layout mapping property `layout_right` gives a layout mapping where the right-most extent is stride one and strides increase right-to-left as the product of extents.
3. If `Extents` is not a (possibly cv-qualified) specialization of `extents`, the program is ill-formed.

```

namespace std {
namespace experimental {
namespace fundamentals_v3 {

struct layout_right {
    template<class Extents>
    class mapping {
    public:
        // [mdspan.layout.right.cons], layout_left::mapping constructors
        constexpr mapping() noexcept;
        constexpr mapping(const mapping& other) noexcept;
        constexpr mapping(mapping&& other) noexcept;
        constexpr mapping(Extents e) noexcept;
        template<class OtherExtents>
            constexpr mapping(const mapping<OtherExtents>& other);

        mapping& operator=() noexcept = default;
        mapping& operator=(const mapping& other) noexcept = default;
        template<class OtherExtents>
            constexpr mapping& operator=(const mapping<OtherExtents>& other);

        // [mdspan.layout.right.ops], layout_right::mapping operations
        Extents get_extents() const noexcept;

        constexpr typename Extents::index_type required_span_size() const noexcept;

        template<class... Indices>
            typename Extents::index_type operator()(Indices... is) const;

        static constexpr bool is_always_unique() noexcept;
        static constexpr bool is_always_contiguous() noexcept;
        static constexpr bool is_always_strided() noexcept;

        constexpr bool is_unique() const noexcept;
        constexpr bool is_contiguous() const noexcept;
        constexpr bool is_strided() const noexcept;

        typename Extents::index_type stride(size_t rank) const noexcept;

        template<class OtherExtents>
            constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
        template<class OtherExtents>
            constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;

    private:
        Extents extents_; // exposition only
    };
};
}

```

26.7.3.1 `layout_right::mapping` constructors [mdspan.layout.right.cons]

```
constexpr mapping() noexcept;
```

- *Effects:* Default-initializes `extents_`.
- *Postconditions:* `get_extents() == Extents()`

```
constexpr mapping(const mapping& other) noexcept;
```

- *Effects*: Initializes `extents_` with `other.extents_`.
- *Postconditions*: `get_extents() == other.get_extents()`.

```
constexpr mapping(mapping&& other) noexcept;
```

- *Effects*: Initializes `extents_` with `move(other.extents_)`.
- *Postconditions*: `get_extents()` returns a copy of an `Extents` that is equal to the copy returned by `other.get_extents()` before the invocation of the move.

```
constexpr mapping(Extents e) noexcept;
```

- *Effects*: Initializes `extents_` with `e`.
- *Postconditions*: `get_extents() == e`.

```
template<class OtherExtents>
constexpr mapping(const mapping<OtherExtents>& other);
```

- *Requires*: `other.get_extents()` meets the requirements for use in the initialization of `extents_`.
- *Effects*: Initializes `extents_` with `other.get_extents()`.
- *Postconditions*: `get_extents() == other.extents_`.
- *Throws*: nothing.

26.7.3.2 `layout_right::mapping` operations [mdspan.layout.right.ops]

```
Extents get_extents() const noexcept;
```

- *Returns*: `extents_`.

```
typename Extents::index_type required_span_size() const noexcept;
```

- *Returns*: The product of `get_extents().extent(r)` for all `r` where $0 \leq r < \text{get_extents().rank}()$

```
template<class... Indices>
typename Extents::index_type operator()(Indices... i) const noexcept;
```

Let `i[k]` denote the k th member of `i...`.

- *Returns*: Equivalent to `offset` in

```
index_type offset = 0;
for(size_t k=0; k<Extents::rank(); ++k)
    offset += i[k]*stride(k);
```


- *Remarks:* This operator shall not participate in overload resolution unless
 - `sizeof...(Indices)==get_extents().rank()`,
 - and `is_convertible_v<Indices, typename Extents::index_type> && ...`

```
static constexpr bool is_always_unique() noexcept;
static constexpr bool is_always_contiguous() noexcept;
static constexpr bool is_always_strided() noexcept;
constexpr bool is_unique() const noexcept;
constexpr bool is_contiguous() const noexcept;
constexpr bool is_strided() const noexcept;
```

- *Returns:* `true`

```
typename Extents::index_type stride(size_t r) const noexcept;
```

- *Returns:* Equivalent to `s` in

```
Extents::index_type s = 1;
for(size_t k=r+1; k<get_extents.rank(); ++k)
    s *= get_extents(k);
```

```
template<class OtherExtents>
constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
```

- *Returns:* `get_extents()==other.get_extents()`.

```
template<class OtherExtents>
constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;
```

- *Returns:* `get_extents()!=other.get_extents()`.

26.7.4 Class `layout_stride` [mdspan.layout.stride]

1. `layout_stride` meets the requirements of layout mapping policy.
2. The layout mapping property `layout_stride` gives a layout mapping where the strides are user defined.
3. If `Extents` is not a (possibly cv-qualified) specialization of `extents`, the program is ill-formed.

```

namespace std {
namespace experimental {
namespace fundamentals_v3 {

struct layout_stride {
    template<class Extents>
    class mapping {
    public:
        // [mdspan.layout.stride.cons], layout_stride::mapping constructors
        constexpr mapping() noexcept;
        constexpr mapping(mapping const& other) noexcept;
        constexpr mapping(mapping&& other) noexcept;
        constexpr mapping(Extents e, array<typename Extents::index_type, Extents::rank()> s) no
except;
        template<class OtherExtents>
            constexpr mapping(const mapping<OtherExtents>& other);

        mapping& operator=() noexcept = default;
        mapping& operator=(const mapping& other) noexcept = default;
        template<class OtherExtents>
            constexpr mapping& operator=(const mapping<OtherExtents>& other);

        // [mdspan.layout.stride.ops], layout_stride::mapping operations
        Extents get_extents() const noexcept;
        array<typename Extents::index_type, Extents::rank()> get_strides() const noexcept;

        constexpr typename Extents::index_type required_span_size() const noexcept;

        template<class... Indices>
            typename Extents::index_type operator()(Indices... is) const;

        static constexpr bool is_always_unique() noexcept;
        static constexpr bool is_always_contiguous() noexcept;
        static constexpr bool is_always_strided() noexcept;

        constexpr bool is_unique() const noexcept;
        constexpr bool is_contiguous() const noexcept;
        constexpr bool is_strided() const noexcept;

        typename Extents::index_type stride(size_t rank) const noexcept;

        template<class OtherExtents>
            constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
        template<class OtherExtents>
            constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;

    private:
        Extents extents_; // exposition only
        array<typename Extents::index_type, Extents::rank()> strides_; // exposition only
    };
};
}}}

```

26.7.4.1 layout_stride::mapping constructors [mdspan.layout.stride.cons]

```
constexpr mapping() noexcept;
```

- Effects: Default-initializes extents_.
- Postconditions: `get_extents() == Extents()` and `get_strides() == array<typename Extents::index_type, Extents::rank()>()`

```
constexpr mapping(const mapping& other) noexcept;
```

- Effects: Initializes `extents_` with `other.extents_`.
- Postconditions: `get_extents()==other.get_extents()` and `get_strides()==other.get_strides()`

```
constexpr mapping(mapping&& other) noexcept;
```

- Effects: Initializes `extents_` with `move(other.extents_)`.
- Postconditions: `get_extents()` returns a copy of an `Extents` that is equal to the copy returned by `other.get_extents()` before the invocation of the move and `get_strides()` returns a copy of an `array<typename Extents::index_type, Extents::rank()>` that is equal to the copy returned by `other.get_strides()` before the invocation of the move

```
constexpr mapping(Extents e, array<typename Extents::index_type, Extents::rank()> s) noexcept;
```

- Requires:
- `s[i]>0` for $0 < i \leq \text{Extents::rank}()$
- there is a permutation of the numbers $0, \dots, \text{Extents::rank}() - 1$ `o(i)` with $0 \leq i < \text{Extents::rank}()$ such that `stride(o(i))>=stride(o(i-1))*get_extents().extent(o(i-1))` for $1 \leq i < \text{Extents::rank}()$
- Effects: Initializes `extents_` with `e` and `strides_` with `s`
- Postconditions: `get_extents()==e` and `get_strides()==s`.
- Throws: nothing

```
template<class OtherExtents>
constexpr mapping(const mapping<OtherExtents>& other);
```

- Requires: `other.get_extents()` meets the requirements for use in the initialization of `extents_`.
- Effects: Initializes `extents_` with `other.get_extents()` and initializes `strides_` with `other.get_strides()`.
- Postconditions: `get_extents()==other.get_extents()` and `get_strides()==other.get_strides()`.
- Throws: nothing.

26.7.4.2 layout_stride::mapping operations [mdspan.layout.stride.ops]

```
Extents get_extents() const noexcept;
```

- Returns: `extents_`.

```
array<typename Extents::index_type, Extents::rank()> get_strides() const noexcept;
```

- Returns: `strides_`.

```
typename Extents::index_type required_span_size() const noexcept;
```

- Returns: The maximum of `get_extents().extent(r)*stride(r)` for all `r` where $0 \leq r < \text{get_extents().rank}()$

```
template <class... Indices>
typename Extents::index_type operator()(Indices... i) const noexcept;
```

- Returns: If `i...` is `i0, i1, i2, ..., ik` (where $k = \text{Extents::rank}() - 1$) and `s = get_strides()`, returns `i0*s[1]+i1*s[2]+...+ik*s[k]`

- Remarks: This operator shall not participate in overload resolution unless
- `sizeof...(Indices)==Extents::rank()`,
- and `is_convertible_v<Indices, typename Extents::index_type> && ...`

```
static constexpr bool is_always_unique() noexcept;
static constexpr bool is_always_strided() noexcept;
constexpr bool is_unique() const noexcept;
constexpr bool is_strided() const noexcept;
```

- Returns: true

```
static constexpr bool is_always_contiguous() noexcept;
```

- Returns: false

```
constexpr bool is_contiguous() const noexcept;
```

- Returns: true if there is a permutation of the numbers `0, ..., Extents::rank()-1` `o(i)` with `0 <= i < Extents::rank()` such that `min(stride(o(i))==1` and `stride(o(i))==stride(o(i-1))*get_extent.extent(o(i-1))` with `1 <= i < Extents::rank()` otherwise returns false

```
typename Extents::index_type stride(size_t r) const noexcept;
```

- Returns: `strides_(r)`

```
template<class OtherExtents>
constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
```

- Returns: `get_extents()==other.get_extents()`.

```
template<class OtherExtents>
constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;
```

- Returns: `get_extents() != other.get_extents()`.

26.7. Accessor [mdspan.accessor]

26.7.1 Accessor requirements [mdspan.accessor.reqs]

1. An *accessor* is a class that converts a pointer and an offset into a reference. *[Note: The intended semantic is that the reference refers to a value at the given offset from the given pointer. —end note]*
2. An *accessor* shall meet the requirements of `DefaultConstructible`, `CopyAssignable`, and the requirements in table 26.7.1.

In Table 26.7.1:

- `A` denotes an accessor.
- `a` denotes a value of type `A`.
- `p` denotes a value of type `A::pointer`.
- `i` denotes an integer.

Table 26.7.2: Accessor requirements

Expression	Return Type	Requirements/Notes
<code>A::value_type</code>		
<code>A::pointer</code>		Requires: <code>A::pointer</code> shall meet the requirements of random access iterator ([<code>random.access.iterators</code>]), and <code>iterator_traits<A::pointer>::value_type</code> shall be exactly <code>A::value_type</code> .
<code>A::reference</code>		Requires: <code>iterator_traits<A::pointer>::reference</code> shall be convertible to <code>A::reference</code>
<code>a(p, i)</code>	<code>A::reference</code>	[Note: <code>basic_mdspan</code> implementations behave as if they use this expression in place of <code>p[i]</code> . - end note]

26.7.2 Class `accessor_basic` [`mdspan.accessor.basic`]

1. `accessor_basic` meets the requirements of accessor.
2. `accessor_basic` gives an accessor that has semantics equivalent to dereferencing a pointer to an array of values.
3. If `T` is not an object type or is an array type, the program is ill-formed.

```

namespace std {
namespace experimental {
namespace fundamentals_v3 {

template<class T>
struct accessor_basic {
    using value_type = T;
    using pointer = T*;
    using reference = T&;

    // [mdspan.accessor.basic.ops], <code data-opaque bs-autolink-syntax='`accessor_basic`'>accessor_basic</code> operations
    constexpr reference operator()(pointer p, ptrdiff_t i) const noexcept;
};

}}}
```

26.7.2.1 `accessor_basic` operations [`mdspan.accessor.basic.ops`]

```
constexpr reference operator()(pointer p, ptrdiff_t i) const noexcept;
```

- Returns: `p[i]`

26.7.2 Class template `basic_mdspan` [`mdspan.basic`]

1. The `basic_mdspan` class template maps a multi-index within a multi-index *domain* to a reference to an element in the *codomain* `span`.
2. The multi-index domain space is the Cartesian product of the extents: `[0, extent(0)) * [0, extent(1)) * ... * [0, extent(rank() - 1))`. Each extent may be statically or dynamically specified.
3. As with `span`, the storage of the objects in the *codomain* `span` of a `basic_mdspan` is owned by some other object.
4. `ElementType` is required to be a complete object type that is not an abstract class type or an array type.
5. `Extents` is required to be a (cv-unqualified) specialization of `extents`.
6. `LayoutPolicy` is required to be a cv-unqualified object type.

7. If `LayoutPolicy` does not meet the layout mapping policy requirements, the program is ill-formed.
8. `Accessor` is required to be a cv-unqualified object type.
9. If `Accessor` does not meet the accessor requirements, or if `Accessor::value_type` is not exactly `ElementType`, the program is ill-formed.

```
namespace std {
namespace experimental {
namespace fundamentals_v3 {

template<class ElementType, class Extents, class LayoutPolicy, class Accessor>
class basic_mdspan {
public:

    // Domain and codomain types
    using layout = LayoutPolicy;
    using mapping = typename layout::template mapping<Extents>;
    using accessor = Accessor;
    using element_type = typename accessor::value_type;
    using value_type = remove_cv_t<element_type>;
    using index_type = ptrdiff_t ;
    using difference_type = ptrdiff_t ;
    using pointer = typename accessor::pointer;
    using reference = typename accessor::reference;

    // [mdspan.basic.cons], basic_mdspan constructors, assignment, and destructor
    constexpr basic_mdspan() noexcept = default;
    constexpr basic_mdspan(const basic_mdspan&) noexcept = default;
    constexpr basic_mdspan(basic_mdspan&&) noexcept;
    template<class... IndexType>
        explicit constexpr basic_mdspan(pointer p, IndexType... dynamic_extents);
    template<class... IndexType>
        explicit constexpr basic_mdspan(const span<element_type>& sp, IndexType... dynamic_exte
nts);
    template<class IndexType, size_t N>
        explicit constexpr basic_mdspan(pointer p, const array<IndexType, N>& dynamic_extents);
    template<class IndexType, size_t N>
        explicit constexpr basic_mdspan(const span<element_type>& sp, const array<IndexType, N>
& dynamic_extents);
    constexpr basic_mdspan(pointer p, const mapping& m);
    constexpr basic_mdspan(const span<element_type>& sp, const mapping& m);
    constexpr basic_mdspan(pointer p, const mapping& m, const accessor& a);
    constexpr basic_mdspan(const span<element_type>& sp, const mapping& m, const accessor& a)
;
    template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class Other
Accessor>
        constexpr basic_mdspan(const basic_mdspan<OtherElementType, OtherExtents, OtherLayoutPo
licy, OtherAccessor>& other);

    ~basic_mdspan() = default;

    constexpr basic_mdspan& operator=(const basic_mdspan&) noexcept = default;
    constexpr basic_mdspan& operator=(basic_mdspan&&) noexcept = default;
    template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class Other
Accessor>
        constexpr basic_mdspan& operator=(const basic_mdspan<OtherElementType, OtherExtents, Ot
herLayoutPolicy, OtherAccessor>& other) noexcept;

    // [mdspan.basic.mapping], basic_mdspan mapping domain multi-index to access codomain ele
ment
    constexpr reference operator[](index_type) const noexcept;
    template<class... IndexType>
        constexpr reference operator()(IndexType... indices) const noexcept;
    template<class IndexType, size_t N>
        constexpr reference operator()(const array<IndexType, N>& indices) const noexcept;
    accessor get_accessor() const;
```

```

// [mdspan.basic.domobs], basic_mdspan observers of the domain multi-index space
static constexpr int rank() noexcept;
static constexpr int rank_dynamic() noexcept;
static constexpr index_type static_extent(size_t) noexcept;

constexpr Extents get_extents() const noexcept;
constexpr index_type extent(size_t) const noexcept;
constexpr index_type size() const noexcept;
constexpr index_type unique_size() const noexcept;

// [mdspan.basic.codomain], basic_mdspan observers of the codomain
constexpr span<element_type> span() const noexcept;

// [mdspan.basic.obs], basic_mdspan observers of the mapping
static constexpr bool is_always_unique() noexcept;
static constexpr bool is_always_contiguous() noexcept;
static constexpr bool is_always_strided() noexcept;

constexpr mapping get_mapping() const noexcept;
constexpr bool is_unique() const noexcept;
constexpr bool is_contiguous() const noexcept;
constexpr bool is_strided() const noexcept;
constexpr index_type stride(size_t) const;

private:
    accessor acc_; // exposition only
    mapping map_; // exposition only
    pointer ptr_; // exposition only
};

}}}

```

26.7.1 basic_mdspan constructors, assignment, and destructor [mdspan.basic.cons] // Mapping domain multi-index to access codomain element

```
constexpr basic_mdspan() noexcept = default;
```

- *Effects:*
 - zero-initializes `ptr_`
 - value-initializes `map_`
 - value-initializes `acc_`
- *Postconditions:*
 - `size() == 0`
 - `get_extents() == Extents()`
 - `get_mapping() == mapping()`

```
constexpr basic_mdspan(const basic_mdspan& other) noexcept = default;
```

- *Effects:*
 - initializes `ptr_` with `other.ptr_`
 - initializes `map_` with `other.map_`
 - initializes `acc_` with `other.acc_`
- *Postconditions:*
 - `size() == other.size()`

- `get_extents()==other.get_extents()`
- `get_mapping()==other.get_mapping()`

```
constexpr basic_mdspan(basic_mdspan&& other) noexcept;
```

- *Effects:*

- initializes `ptr_` with `move(other.ptr_)`
- initializes `map_` with `move(other.map_)`
- initializes `acc_` with `move(other.acc_)`

```
template<class... IndexType>
explicit constexpr basic_mdspan(pointer ptr, IndexType... dynamic_extents);
```

- *Requires:* `[ptr, ptr+mapping(Extents(dynamic_extents...)).required_span_size()]` shall be a valid range.
- *Effects:*
 - initializes `ptr_` with `ptr`
 - initializes `map_` with `Extents(dynamic_extents...)`
 - value-initializes `acc_`
- *Postconditions:*
 - `get_extents()==Extents(dynamic_extents...)`
 - `get_mapping()==mapping(Extents(dynamic_extents...))`
- *Remarks:* This constructor will not participate in overload resolution unless:
 - `(is_convertible_v<IndexType, index_type> && ...)`,
 - `sizeof...(dynamic_extents)==rank_dynamic()`, and
 - `is_constructible_v<mapping, Extents>`.
- *Throws:* Nothing.

```
template<class... IndexType>
explicit constexpr basic_mdspan(const span<element_type>& sp, IndexType... dynamic_extents);
```

- *Requires:* `sp.size()==mapping(Extents(dynamic_extents...)).required_span_size()`
- *Effects:*
 - initializes `ptr_` with `sp.data()`
 - value-initializes `map_`
 - value-initializes `acc_`
- *Postconditions:*
 - `get_extents()==Extents(dynamic_extents...)`
 - `get_mapping()==mapping(Extents(dynamic_extents...))`
- *Remarks:* This constructor will not participate in overload resolution unless:
 - `(is_convertible_v<IndexType, index_type> && ...)`,

- `sizeof...(dynamic_extents)==rank_dynamic()`,
- `is_constructible_v<mapping, Extents>`.
- *Throws:* Nothing.

```
template<class IndexType, size_t N>
    explicit constexpr basic_mdspan(pointer p, const array<IndexType, N>& dynamic_extents);
```

- *Requires:* `[ptr, ptr+mapping(Extents(dynamic_extents)).required_span_size()]` shall be a valid range.
- *Effects:* Equivalent to `basic_mdspan(p, dynamic_extents[Rs]...)`, with `Rs...` from `index_sequence<Rs...>` matching `make_index_sequence<N>`.
- *Remarks:* This constructor does not participate in overload resolution unless
 - `IndexType` is convertible to `index_type`, and
 - `N==rank_dynamic()`, and
 - `is_constructible_v<mapping, Extents>`.
- *Throws:* Nothing.

```
template<class IndexType, size_t N>
    explicit constexpr basic_mdspan(const span<element_type>& sp, const array<IndexType, N>&
    dynamic_extents);
```

- *Requires:* `sp.size()==mapping(Extents(dynamic_extents)).required_span_size()`
- *Effects:* Equivalent to `basic_mdspan(sp.data(), dynamic_extents[Rs]...)`, with `Rs...` from `index_sequence<Rs...>` matching `make_index_sequence<N>`.
- *Remarks:* This constructor does not participate in overload resolution unless
 - `IndexType` is convertible to `index_type`,
 - `N==rank_dynamic()`,
 - `is_constructible_v<mapping, Extents>`.
- *Throws:* Nothing.

```
constexpr basic_mdspan(pointer p, const mapping& m);
```

- *Requires:* `[ptr, ptr+m.required_span_size()]` shall be a valid range.
- *Effects:*
 - initializes `ptr_` with `p`
 - initializes `map_` with `m`
 - value-initializes `acc_`
- *Postconditions:*
 - `get_extents()==m.get_extents()`
 - `get_mapping()==m`
- *Throws:* Nothing.

```
constexpr basic_mdspan(const span<element_type>& sp, const mapping& m);
```

- *Requires:* `sp.size()==m.required_span_size()`
- *Effects:* Equivalent to `basic_mdspan(sp.data(), m)`
- *Remarks:* This constructor does not participate in overload resolution unless `is_constructible_v<mapping, Extents>`.
- *Throws:* Nothing.

```
constexpr basic_mdspan(pointer p, const mapping& m, const accessor& a);
```

- *Requires:* `[ptr, ptr+m.required_span_size())` shall be a valid range.
- *Effects:*
 - initializes `ptr_` with `p`
 - initializes `map_` with `m`
 - initializes `acc_` with `a`
- *Postconditions:*
 - `get_extents()==m.get_extents()`
 - `get_mapping()==m`
- *Throws:* Nothing.

```
constexpr basic_mdspan(const span<element_type>& sp, const mapping& m, const accessor& a);
```

- *Requires:* `sp.size()==m.required_span_size()`
- *Effects:* Equivalent to `basic_mdspan(sp.data(), m, a)`
- *Remarks:* This constructor does not participate in overload resolution unless
 - `is_constructible_v<mapping, Extents>`.
- *Throws:* Nothing.

```
template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class OtherAccessor>
constexpr basic_mdspan(const basic_mdspan<OtherElementType, OtherExtents, OtherLayoutPolicy, OtherAccessor>& other);
```

- *Requires:*
 - For all `r` in the range `[0, rank())`, if `other.static_extent(r)==dynamic_extent` or `static_extent(r)==dynamic_extent`, then `other.extent(r)==extent(r)`
- *Effects:*
 - initializes `ptr_` with `other.ptr_`
 - initializes `map_` with `other.map_`
 - initializes `acc_` with `other.acc_`
- *Postconditions:*
 - `get_extents()==Extents(other.get_extents())`
 - `get_mapping()==mapping(other.get_mapping())`

- *Remarks:* This constructor will not participate in overload resolution unless all of the following conditions are met:
 - `OtherElementType(*)[]` is convertible to `ElementType(*)[]`,
 - `OtherLayoutPolicy::template mapping<OtherExtents>` is convertible to `mapping`
 - `OtherAccessor` is convertible to `Accessor`, and
 - `OtherAccessor::pointer` is convertible to `pointer`.
- *Throws:* Nothing.

```
template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class OtherAccessor>
constexpr basic_mdspan& operator=(const basic_mdspan<OtherElementType, OtherExtents, OtherLayoutPolicy, OtherAccessor>& other);
```

- *Requires:*
 - For all `r` in the range `[0, rank())`, if `other.static_extent(r)==dynamic_extent` or `static_extent(r)==dynamic_extent`, then `other.extent(r)==extent(r)`
- *Effects:*
 - assigns `other.ptr_` to `ptr_`.
 - assigns `other.map_` to `map_`
 - assigns `other.acc_` to `acc_`
- *Postconditions:*
 - `get_extents()==Extents(other.get_extents())`
 - `get_mapping()==mapping(other.get_mapping())`
- *Remarks:* This constructor will not participate in overload resolution unless all of the following conditions are met:
 - `OtherElementType(*)[]` is assignable to `ElementType(*)[]`,
 - `OtherLayoutPolicy::template mapping<OtherExtents>` is assignable to `mapping`
 - `OtherAccessor` is assignable to `Accessor`, and
 - `OtherAccessor::pointer` is assignable to `pointer`.
- *Throws:* Nothing.

26.7.2 `basic_mdspan` mapping domain multi-index to access codomain element [mdspan.basic.mapping]

```
constexpr reference operator[](index_type i) const;
```

- *Effects:* Equivalent to `return (*this)(i);`.
- *Remarks:* This operator does not participate in overload resolution unless `rank()==1`

```
template<class... IndexType>
constexpr reference operator()(IndexType... indices) const;
```

- *Requires:* `0 <= array<index_type, sizeof...(indices)>{indices...}[r] < extent(r)` for all `r` in the range `[0, rank())`.
- *Effects:* Equivalent to `return acc_(ptr_, indices...);`

- *Remarks:* This operator does not participate in overload resolution unless
 - `is_convertible_v<IndexType, index_type> && ...`
 - `sizeof...(IndexType)==rank()`
- *Throws:* nothing.

```
template<class IndexType, size_t N>
constexpr reference operator()(const array<IndexType, N>& indices) const;
```

- *Effects:* Equivalent to `return std::apply(*this, indices);`
- *Remarks:* This operator does not participate in overload resolution unless
 - `is_convertible_v<IndexType, index_type>`
 - `rank()==N`
- *Throws:* nothing.

```
accessor get_accessor() const;
```

- *Returns:* `acc_`.

26.7.3 `basic_mdspan` observers of the domain multi-index space [`mdspan.basic.domobs`]

```
static constexpr int rank() noexcept;
```

- *Returns:* `Extents::rank()`.

```
static constexpr int rank_dynamic() noexcept;
```

- *Returns:* `Extents::rank_dynamic()`.

```
static constexpr index_type static_extent(size_t r) noexcept;
```

- *Returns:* `Extents::static_extent(r)`.

```
constexpr Extents get_extents() const noexcept;
```

- *Returns:* `get_mapping().get_extents()`.

```
constexpr index_type extent(size_t r) const noexcept;
```

- *Returns:* `get_extents().extent(r)`.

```
constexpr index_type size() const noexcept;
```

- *Returns:* Product of `extent(r)` for all `r` where $0 \leq r < \text{get_extents}().\text{rank}()$.

```
constexpr index_type unique_size() const noexcept;
```

- *Returns:* The number of unique elements in the codomain. *[Note: If `get_mapping().is_unique()` is true, this is identical to `size()` —end note]*

26.7.4.3 `basic_mdspan` observers of the codomain [`mdspan.basic.codomain`]

```
constexpr span<element_type> span() const noexcept;
```

- *Returns:* The codomain `span`.

26.7.4.4 `basic_mdspan` observers of the mapping [`mdspan.basic.obs`]

```
static constexpr bool is_always_unique() noexcept;
```

- *Returns:* `mapping::is_always_unique()`

```
static constexpr bool is_always_contiguous() noexcept;
```

- *Returns:* `mapping::is_always_contiguous()`

```
static constexpr bool is_always_strided() noexcept;
```

- *Returns:* `mapping::is_always_strided()`

```
constexpr mapping get_mapping() const noexcept;
```

- *Returns:* `map_`

```
constexpr bool is_unique() const noexcept;
```

- *Returns:* `get_mapping().is_unique()`

```
constexpr bool is_contiguous() const noexcept;
```

- *Returns:* `get_mapping().is_contiguous()`

```
constexpr bool is_strided() const noexcept;
```

- *Returns:* `get_mapping().is_strided()`

```
constexpr index_type stride(size_t r) const;
```

- Returns: `get_mapping().stride(r)`

26.7. subspan [mdspan.subspan]

```
namespace std {
namespace experimental {
namespace fundamentals_v3 {

    // [mdspan.subspan], subspan creation
    template<class ElementType, class Extents, class LayoutPolicy,
            class Accessor, class... SliceSpecifiers>
        basic_mdspan<ElementType, E /* see-below */, layout_stride, Accessor>
            subspan(const basic_mdspan<ElementType, Extents, LayoutPolicy, Accessor>& src, SliceS
pecifiers ... slices) noexcept;

}}}
```

`subspan` creates a `basic_mdspan` that is a view on a (potentially trivial) subset of another `basic_mdspan`. The `SliceSpecifier` parameters indicate the subset that the return value references. Let `slices[r]` denote the r th value in the parameter pack `slices`. The second template parameter of the return type is `E`, a specialization of `extents`.

Let C be the number of parameters in `SliceSpecifiers` which are convertible to `ptrdiff_t`. Let $ranges[r]$ denote the r th value in the parameter pack `slices`, which is not convertible to `ptrdiff_t`. Let sub be the return value of `subspan`. Let $first[r]$ denote the r th lower bound of `slices[r]`.

- If `slices[r]` is convertible to `ptrdiff_t` `first[r]==slices[r]`
- If `slices[r]` is convertible to `pair<ptrdiff_t,ptrdiff_t>` `first[r]==pair<ptrdiff_t,ptrdiff_t>(slices[r]).first()`
- If `slices[r]` is convertible to `alltype_t` `first[r]==0`

Let $last[r]$ denote the r th upper bound of `slices[r]`.

- If `slices[r]` is convertible to `ptrdiff_t` `last[r]==slices[r]+1`
- If `slices[r]` is convertible to `pair<ptrdiff_t,ptrdiff_t>` `last[r]==pair<ptrdiff_t,ptrdiff_t>(slices[r]).second()`
- If `slices[r]` is convertible to `alltype_t` `last[r]==src.extent(r)`

Requires:

- `sizeof(slices...)==Extents::rank()`
- `slices[r]` is convertible to `ptrdiff_t`, `pair<ptrdiff_t,ptrdiff_t>` or `alltype_t`
- $0 \leq first[r] < src.extent(r)$
- $0 < last[r] \leq src.extent(r)$
- `LayoutPolicy` is `layout_right`, `layout_left`, or `layout_stride`

Postcondition:

- `E::rank()==Extents::rank()-C`
- If $ranges[r]$ is the n th value of `ranges` convertible to `alltype_t` or `pair<ptrdiff_t,ptrdiff_t>` and $slices[k]$ is the n th value of `slices` convertible to `alltype_t` or `pair<ptrdiff_t,ptrdiff_t>` then `sub.extent(r)==last[k]-first[k]` and `sub.stride(r)==src.stride(k)`

- `sub.ptr_==src.ptr_+first[0]*src.stride(0)+...+first[Extents::rank()-1]*src.stride(Extents::rank()-1)`

Note: it is quality of implementation whether static extents are preserved if possible.

- Examples:

```
// Create a mapping
typedef extents<3,dynamic_extent,7> Extents3D;
layout_right::template mapping<Extents3D> map_right(10);

// Allocate a basic_mdspan
int* ptr = new int[3*8*10];
basic_mdspan<int,Extents3D,layout_right> a(ptr,map_right);

// Initialize the span
for(int i0=0; i0<a.extent(0); i0++)
    for(int i1=0; i1<a.extent(1); i1++)
        for(int i2=0; i2<a.extent(2); i2++)
            a(i0,i1,i2) = 10000*i0+100*i1+i2;

// Create Subspan
auto a_sub = subspan(a,1,std::pair<int,int>(4,6),std::pair<int,int>(1,6));

// Print values of subspan
for(int i0=0; i0<a_sub.extent(0); i0++) {
    for(int i1=0; i1<a_sub.extent(1); i1++)
        std::cout << a_sub(i0,i1) << " ";
    std::cout << std::endl;
}

/* Output
10401 10402 10403 10404 10405
10501 10502 10503 10504 10505
*/
```

§ 5. Next Steps

- Wording editing as per guidance from LWG.

§ 6. Related Work

[LEWG issue](#)

Previous paper:

- [\[N4355\]](#)

P0860 : Access Policy Generating Proxy Reference

The [reference](#) type may be a proxy for accessing an [element_type](#) object. For example, the *atomic* [AccessorPolicy](#) in **P0860** defines `AccessorPolicy::template accessor<T>::reference` to be `atomic_ref<T>` from **P0019**.

Related papers:

- **P0122** : span: bounds-safe views for sequences of objects The [mdspan](#) codomain concept of *span* is well-aligned with this paper.
- **P0367** : Accessors The P0367 Accessors proposal includes polymorphic mechanisms for accessing the memory an object or span of objects. The [AccessPolicy](#) extension point in this proposal is intended to include such memory access properties.
- **P0331** : Motivation and Examples for Multidimensional Array

- **P0332** : Relaxed Incomplete Multidimensional Array Type Declaration
- **P0454** : Wording for a Minimal `mdspan` Included proposed modification of `span` to better align `span` with `mdspan`.
- **P0546** : Preparing `span` for the future Proposed modification of `span`
- **P0856** : Restrict access property for `mdspan` and `span`
- **P0860** : atomic access policy for `mdspan`
- **P0900** : An Ontology of Properties for `mdspan`

§ 7. TODO

- Design questions:
 - ~~Need a way to construct `span` with non-default-initialized `mapping` and `accessor`~~ Added, please review
 - Should we be constraining things with `is_integral_v<IndexType>` or `is_convertible_v<IndexType, index_type>`?
 - ~~Extents and/or mapping conversion?~~ Done, please review
 - ~~Constrain constructors of `basic_mdspan` based on `Mapping` and `Accessor` constraints.~~ Done, please review
 - `unique_size()` done

§ References

§ Informative References

[N4355]

Carter Edwards. [Shared Multidimensional Arrays with Polymorphic Layout](https://wg21.link/n4355). 4 February 2015. URL: <https://wg21.link/n4355>

[P0900r0]

David S. Hollman. [An Ontology for Properties of `mdspan`](https://wg21.link/p0900r0). 12 February 2018. URL: <https://wg21.link/p0900r0>